

Reflexion – Teilaufgabe 3

In der Planung wurde der Fokus eher auf das Speichern der Daten vom Client gelegt und es gab einen generellen Mangel an Klassen, die sich auf die Verarbeitungslogik konzentriert haben. In meiner Implementierung habe ich die Map-Klasse in eine Klasse zur Kartenerstellung (mit Logik zur Kartenkombination) und eine GameMap-Klasse aufgeteilt, in der die Spielkarte gespeichert wird. Außerdem musste ich die Logik dafür implementieren, dass eine bestimmte Ansicht der Spielkarte angezeigt wird, je nachdem, welcher Spieler gerade den "GameState" anfordert, was ich in der Planungsphase überhaupt nicht in Betracht gezogen hatte, was aber zu den Dingen gehört, die bei der Implementierung relativ viel Zeit in Anspruch genommen hat.

Außerdem habe ich zusätzlich eine Move-Klasse hinzugefügt, die sich ausschließlich mit der Bewegungslogik des Spielers befasst, was ebenfalls im Entwurf gefehlt hat. Für die Kartenvalidierung und viele der anderen Geschäftsregeln habe ich ein Interface erstellt, das von verschiedenen Klassen implementiert wird, je nachdem, welche Geschäftsregel überprüft wird, was die Erweiterbarkeit und Wartbarkeit beim späteren Einfügen von neuen Geschäftsregeln (z. B. für den Move-Endpunkt) deutlich erhöht hat.

Y-Statements

1. Im Kontext der vom Client empfangenen Spielkarte habe ich mich dafür entschieden, die Map-Klasse in eine GameMap- und GameMapCreation-Klasse aufzuteilen, um das Single-Responsibility-Prinzip einzuhalten und die Speicherung und Logik der Spielkarte zu trennen, unter Inkaufnahme, dass dies im Endeffekt zu mehr Code und eventuell mehr Kopplung führen kann.
2. Im Kontext der Geschäftsregeln habe ich mich angesichts der vielen Geschäftsregeln, die vom Server überprüft werden müssen, für die Erstellung eines IBusinessRule-Interface entschieden, um das Open-Closed-Prinzip einzuhalten und die Wartbarkeit und Erweiterbarkeit des Codes zu verbessern, unter Inkaufnahme, dass dies die Anzahl der Klassen letztendlich erhöhen wird.
3. Im Kontext der Implementierung der Bewegungslogik des Players habe ich mich dafür entschieden, eine Move-Klasse zu erstellen, um die Bewegungsberechnungen und andere Bewegungslogik angesichts der unübersichtlichen und komplizierten Player-Klasse zu behandeln, um sicherzustellen, dass das Single-Responsibility-Prinzip für die Player-Klasse eingehalten wird und um die Klasse möglichst einfach zu halten.